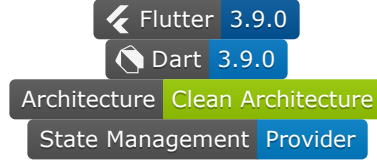




EventUp - Developer Documentation



EventUp Developer Documentation

[Architecture](#) • [Setup](#) • [Contributing](#) • [API](#)



İçindekiler

- [Proje Genel Bakış](#)
 - [Mimari Genel Bakış](#)
 - [Mimari Katmanları](#)
 - [Veri Akışı](#)
 - [Hata Yönetimi](#)
 - [Dependency Injection](#)
 - [Geliştirme Kurulumu](#)
 - [Dependency Injection](#)
 - [State Management](#)
 - [BaseProvider Yapısı](#)
 - [Provider Mixins](#)
 - [Result Pattern](#)
 - [API ve Servis Katmanı](#)
 - [Network Layer](#)
 - [HTTP Client](#)
 - [Interceptors](#)
 - [Kod Yapısı](#)
 - [Geliştirme Rehberi](#)
 - [Test Stratejisi](#)
 - [Build ve Deployment](#)
-



Proje Genel Bakış

EventUp, kullanıcıların ilgi alanlarına göre etkinlik keşfedip katılabileceği, diğer katılımcılarla anlık mesajlaşabileceği modern bir Flutter mobil uygulamasıdır.

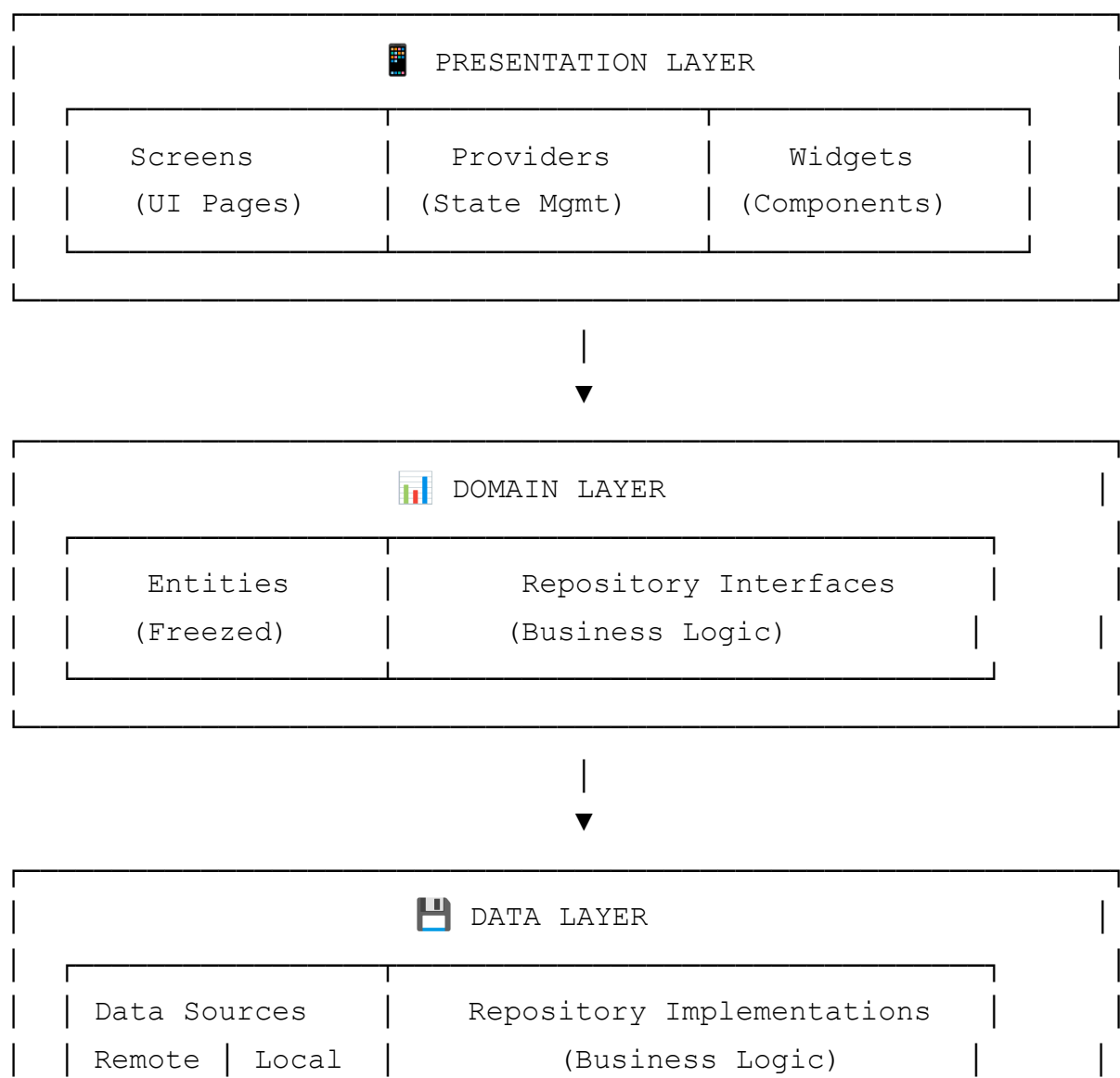
Ana Özellikler

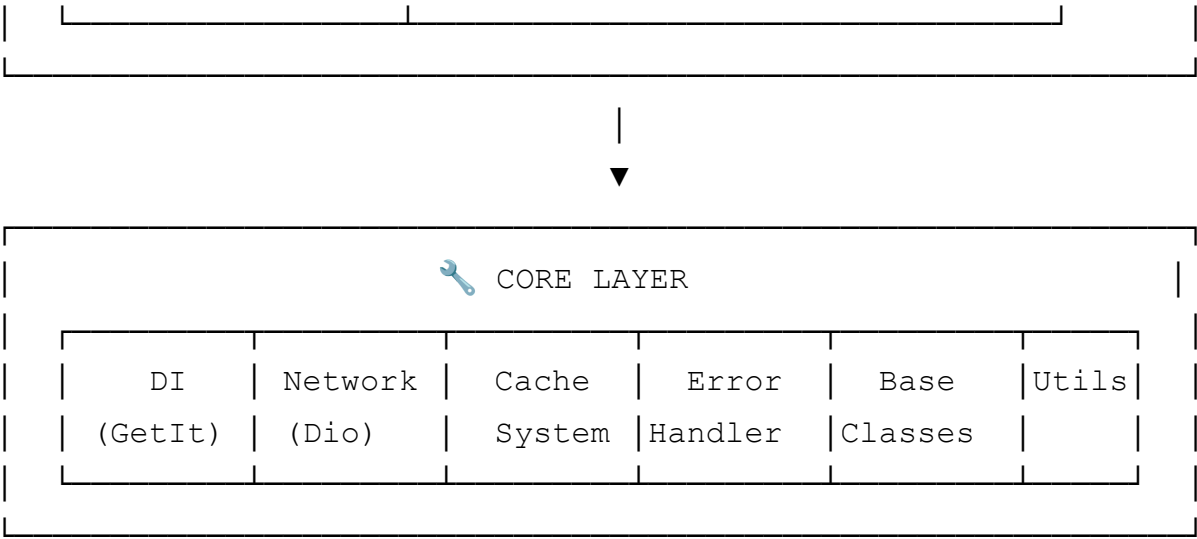
- ✓ Güvenli kullanıcı kimlik doğrulama
- ✓ Etkinlik keşfetme ve katılım
- ✓ Real-time mesajlaşma (SignalR)
- ✓ Profil yönetimi ve ilgi alanları
- ✓ Modern UI/UX tasarım

Mimari Genel Bakış

Proje **Clean Architecture** prensiplerine uygun şekilde tasarlanmıştır ve **SOLID** prensiplerini takip eder.

Mimari Katmanları





Detaylı Katman Açıklamaları

📱 Presentation Layer (UI & State Management)

- **Providers:** State management için Provider pattern
- **Screens:** Kullanıcı arayüzü ekranları
- **Widgets:** Yeniden kullanılabilir UI bileşenleri

```
// Provider Örneği
@injectable
class EventsProvider extends BaseProvider with ListProviderMixin<Event> {
  Future<void> loadEvents() async {
    await execute(
      () => _repository.getEvents(),
      onSuccess: setItems,
    );
  }
}
```

📊 Domain Layer (Business Logic)

- **Entities:** Immutable business objects (Freezed ile)
- **Repository Interfaces:** Business logic abstraction

```
// Entity Örneği
@freezed
class Event with _$Event {
  const factory Event({
    required String id,
    required String title,
```

```

        required DateTime date,
    }) = _Event;
}

```

Data Layer (Data Access)

- **Data Sources:** API calls ve local storage
- **Repository Implementations:** Data operations

```

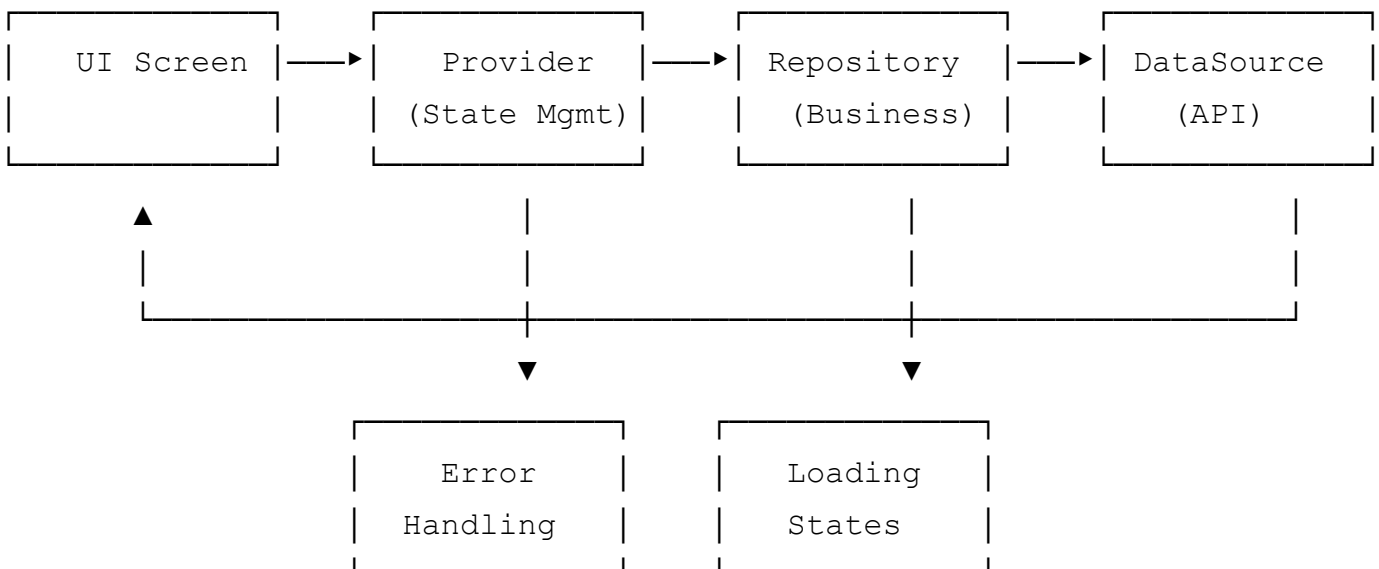
// Repository Implementation
@LazySingleton()
class EventsRepository {
    Future<Result<List<Event>>> getEvents() async {
        // Business logic implementation
    }
}

```

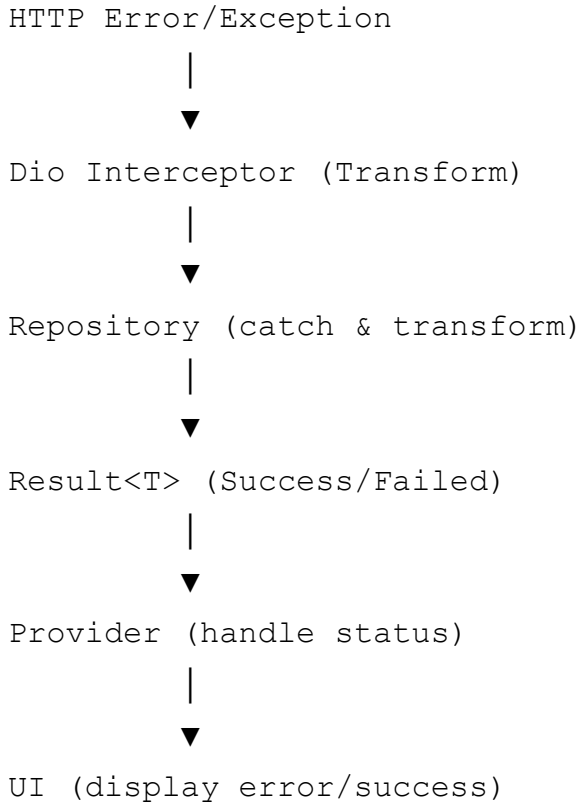
Core Layer (Infrastructure)

- **DI:** Dependency injection (GetIt + Injectable)
- **Network:** HTTP client (Dio + Interceptors)
- **Cache:** Memory caching system
- **Error:** Exception & failure handling
- **Base:** Base classes ve mixins
- **Utils:** Utilities ve helpers

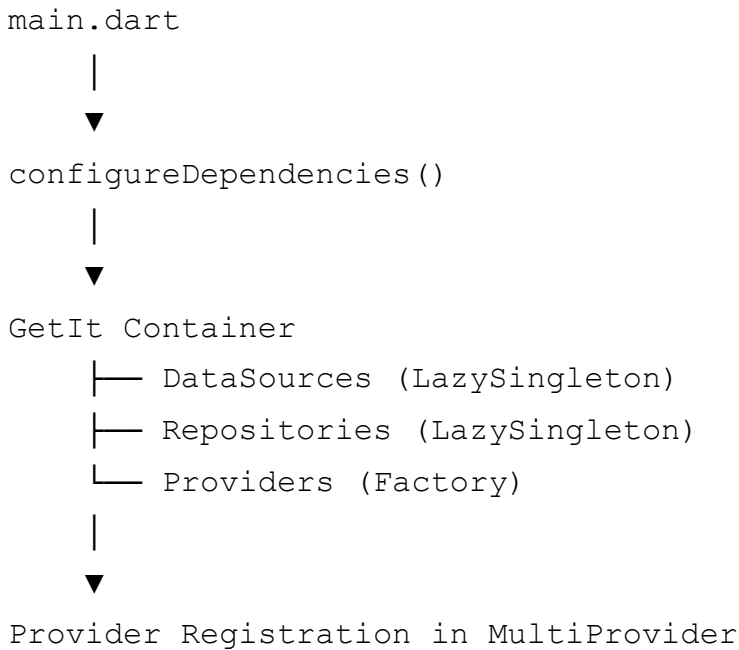
Veri Akışı Diyagramı



Hata Yönetimi Akışı



Dependency Injection Yapısı



Geliştirme Kurulumu

Gereksinimler

- **Flutter:** ^3.9.0

- **Dart:** ^3.9.0
- **VS Code / Android Studio**
- **Git**

Kurulum Adımları

1. Repository'yi klonlayın

```
git clone <repository-url>
cd event_up_app
```

2. Dependencies'leri yükleyin

```
flutter pub get
```

3. Code generation'ı çalıştırın

```
flutter packages pub run build_runner build
```

4. Uygulamayı çalıştırın

```
flutter run
```

Environment Configuration

Uygulama environment-specific ayarları `lib/config/environment.dart` dosyasında yönetir:

```
// Development
flutter run --dart-define=API_URL=http://localhost:5001/api

// Production
flutter run --dart-define=API_URL=https://api.eventup.com
```



Dependency Injection

Proje, **GetIt** ve **Injectable** kullanarak güçlü bir DI sistemi implement eder.

Setup

```
// lib/core/di/injection.dart
final GetIt getIt = GetIt.instance;

@InjectableInit()
Future<void> configureDependencies() async {
  await getIt.init();
}
```

Dependency Registration

```
// DataSource Registration
@LazySingleton()
class AuthRemoteDataSource {
  // Implementation
}

// Repository Registration
@LazySingleton()
class AuthRepository {
  // Implementation
}

// Provider Registration
@injectable
class AuthProvider extends BaseProvider {
  // Implementation
}
```

Usage in Code

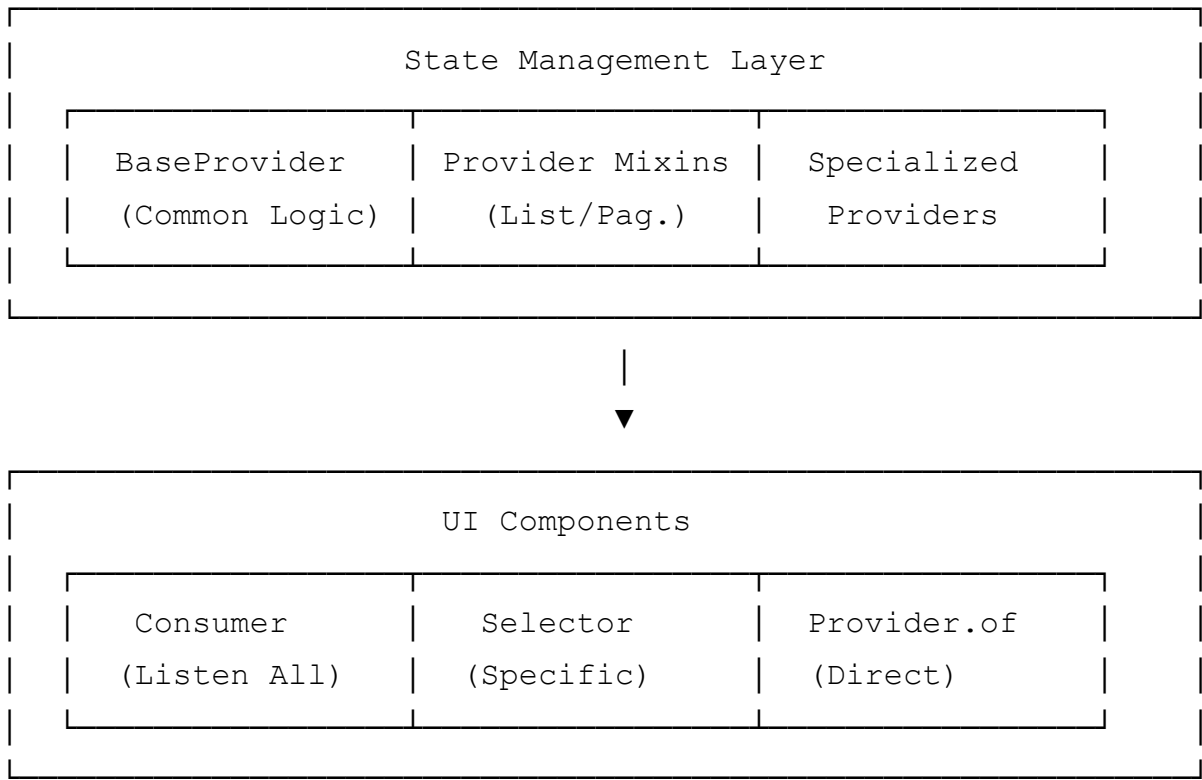
```
// Provider injection in main.dart
ChangeNotifierProvider<AuthProvider>(
  create: (_) => getIt<AuthProvider>(),
)

// Direct injection
final authProvider = getIt<AuthProvider>();
```

State Management

Proje **Provider** pattern kullanır ve `BaseProvider` ile tutarlılık sağlar.

State Management Mimarisi



BaseProvider Yapısı

`BaseProvider` , tüm provider'lar için ortak state management mantığını sağlar:

```
abstract class BaseProvider extends ChangeNotifier {
  bool _isLoading = false;
  Failure? _failure;

  // State getters
  bool get isLoading => _isLoading;
  Failure? get failure => _failure;
  String? get errorMessage => _failure?.message;
  bool get hasError => _failure != null;

  // State setters
  @protected
  void setLoading(bool value) {
    _isLoading = value;
```



```

        notifyListeners();
    }

    @protected
    void setFailure(Failure? failure) {
        _failure = failure;
        notifyListeners();
    }

    // Execute operation with automatic state management
    @protected
    Future<T?> execute<T>(
        Future<Result<T>> Function() operation, {
        void Function(T data)? onSuccess,
        void Function(Failure failure)? onFailure,
        bool showLoading = true,
    }) async {
        if (showLoading) setLoading(true);
        setFailure(null);

        try {
            final result = await operation();
            return result.fold(
                onSuccess: (data) {
                    onSuccess?.call(data);
                    return data;
                },
                onFailure: (failure) {
                    setFailure(failure);
                    onFailure?.call(failure);
                    return null;
                },
            );
        } catch (e) {
            final failure = UnknownFailure(message: e.toString());
            setFailure(failure);
            onFailure?.call(failure);
            return null;
        } finally {
            if (showLoading) setLoading(false);
        }
    }
}

```

Provider Mixins

ListProviderMixin

Liste yönetimi için kullanılır:

```
mixin ListProviderMixin<T> on BaseProvider {
    List<T> _items = [];

    List<T> get items => List.unmodifiable(_items);
    bool get isEmpty => _items.isEmpty;
    bool get hasItems => _items.isNotEmpty;
    int get itemCount => _items.length;

    @protected
    void setItems(List<T> items) {
        _items = items;
        notifyListeners();
    }

    @protected
    void addItem(T item) {
        _items.add(item);
        notifyListeners();
    }

    @protected
    void removeItem(T item) {
        _items.remove(item);
        notifyListeners();
    }

    @protected
    void clearItems() {
        _items = [];
        notifyListeners();
    }
}
```

PaginationProviderMixin

Sayfalama desteği için kullanılır:

```

mixin PaginationProviderMixin<T> on BaseProvider {
  int _currentPage = 1;
  int _totalPages = 1;
  bool _hasMore = true;
  List<T> _items = [];

  int get currentPage => _currentPage;
  int get totalPages => _totalPages;
  bool get hasMore => _hasMore;
  List<T> get items => List.unmodifiable(_items);

  @protected
  void setPaginatedData({
    required List<T> items,
    required int currentPage,
    required int totalPages,
    bool append = false,
  }) {
    _currentPage = currentPage;
    _totalPages = totalPages;
    _hasMore = currentPage < totalPages;

    if (append) {
      _items.addAll(items);
    } else {
      _items = items;
    }
    notifyListeners();
  }
}

```

Result Pattern

Error handling için functional approach:

```

sealed class Result<T> {
  const Result();

  bool get isSuccess => this is Success<T>;
  bool get isFailure => this is Failed<T>;

  T? get dataOrNull => isSuccess ? (this as Success<T>).value : null;
}

```

```

Failure? get failureOrNull => isFailure ? (this as Failed<T>).failure :

R fold<R>({
    required R Function(T data) onSuccess,
    required R Function(Failure failure) onFailure,
}) {
    if (isSuccess) {
        return onSuccess((this as Success<T>).value);
    } else {
        return onFailure((this as Failed<T>).failure);
    }
}

final class Success<T> extends Result<T> {
    const Success(this.value, {this.message});
    final T value;
    final String? message;
}

final class Failed<T> extends Result<T> {
    const Failed(this.failure);
    final Failure failure;
}

```

Provider Örneği

```

@injectable
class EventsProvider extends BaseProvider with ListProviderMixin<Event> {
    EventsProvider(this._eventsRepository);
    final EventsRepository _eventsRepository;

    List<Event> get events => items;

    Future<void> loadEvents({
        required String token,
        String? search,
        String? location,
    }) async {
        await execute(
            () => _eventsRepository.getFilteredEvents(
                token: token,

```

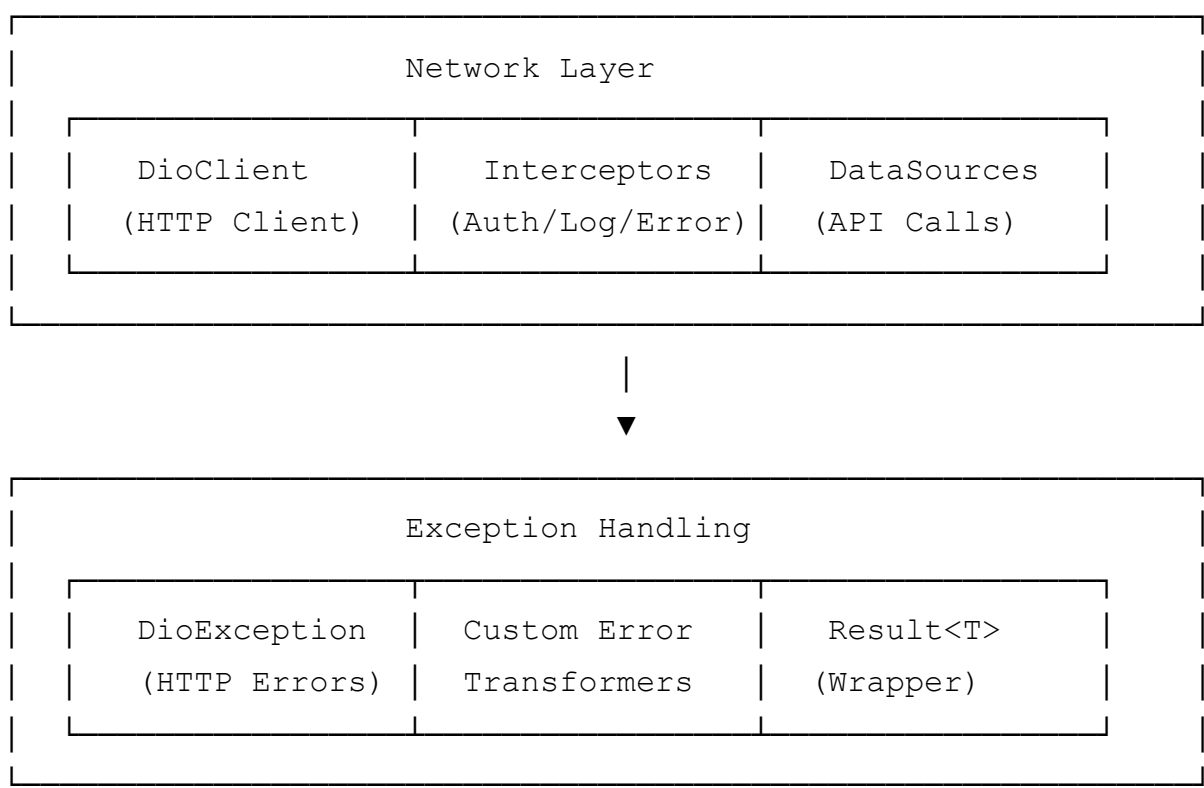
```

        search: search,
        location: location,
    ),
    onSuccess: setItems,
);
}
}

```

API ve Servis Katmanı

Network Layer Mimarisi



HTTP Client (DioClient)

Merkezi HTTP client yapılandırması:

```

@lazySingleton
class DioClient {
    late final Dio _dio;
    Dio get dio => _dio;

    DioClient() {
        _dio = Dio(_baseOptions);
    }
}

```

```

    _setupInterceptors();
}

BaseOptions get _baseOptions => BaseOptions(
  baseUrl: Environment.apiUrl,
  connectTimeout: AppConstants.connectionTimeout,
  receiveTimeout: AppConstants.receiveTimeout,
  sendTimeout: AppConstants.apiTimeout,
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
);

void _setupInterceptors() {
  _dio.interceptors.addAll([
    _LoggingInterceptor(), // Request/Response logging
    _AuthInterceptor(),    // Token management
    _ErrorInterceptor(),   // Error transformation
  ]);
}

// Token management
void setAuthToken(String? token) {
  if (token != null && token.isNotEmpty) {
    _dio.options.headers['Authorization'] = 'Bearer $token';
  } else {
    _dio.options.headers.remove('Authorization');
  }
}
}

```

Interceptors

1. Logging Interceptor

Request ve response'ları loglar:

```

class _LoggingInterceptor extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
    AppLogger.network(
      '${options.method} ${options.path}',

```

```

        method: options.method,
        url: options.uri.toString(),
    );

    if (options.data != null) {
        AppLogger.log('Request Body: ${options.data}', tag: 'DioClient');
    }
    super.onRequest(options, handler);
}

@override
void onResponse(Response response, ResponseInterceptorHandler handler) {
    AppLogger.success(
        'Response ${response.statusCode}: ${response.requestOptions.path}',
        tag: 'DioClient',
    );
    super.onResponse(response, handler);
}
}

```

2. Auth Interceptor

Authentication ve token yönetimi:

```

class _AuthInterceptor extends Interceptor {
    @override
    Future<void> onError(DioException err, ErrorInterceptorHandler handler) {
        // 401 (Unauthorized) durumunda token refresh
        if (err.response?.statusCode == 401) {
            AppLogger.warning('401 Unauthorized - Attempting token refresh');

            // Token refresh logic burada implementasyon yapılabilir
            // 1. Refresh token'ı local storage'dan al
            // 2. Refresh endpoint'ini çağır
            // 3. Yeni token'ı güncelle
            // 4. Original request'i retry et
        }
        super.onError(err, handler);
    }
}

```

3. Error Interceptor

HTTP hatalarını custom exception'lara çevirir:

```
class _ErrorInterceptor extends Interceptor {
  @override
  void onError(DioException err, ErrorInterceptorHandler handler) {
    switch (err.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.receiveTimeout:
        throw TimeoutException(message: 'Request timeout');

      case DioExceptionType.connectionError:
        throw NetworkException(message: 'Network connection failed');

      case DioExceptionType.badResponse:
        _handleResponseError(err);
        break;

      default:
        throw UnknownException(message: 'Unknown error occurred');
    }
    super.onError(err, handler);
  }

  void _handleResponseError(DioException err) {
    final statusCode = err.response?.statusCode;
    final data = err.response?.data;

    String errorMessage = 'Server error occurred';
    if (data != null && data is Map<String, dynamic>) {
      errorMessage = data['message'] ?? errorMessage;
    }

    switch (statusCode) {
      case 400:
        throw ValidationException(message: errorMessage, statusCode: statusCode);
      case 401:
        throw AuthenticationException(message: errorMessage, statusCode: statusCode);
      case 403:
        throw AuthorizationException(message: errorMessage, statusCode: statusCode);
      case 404:
        throw NotFoundException(message: errorMessage, statusCode: statusCode);
      case 500:
        throw ServerException(message: errorMessage, statusCode: statusCode);
      default:
    }
  }
}
```



```

        throw UnknownException(message: errorMessage, statusCode: statusC
    }
}
}

```

Repository Pattern

```

// Repository Interface (Domain Layer)
abstract class AuthRepository {
    Future<Result<UserData>> login({
        required String userNameOrEmail,
        required String password,
    });
}

// Repository Implementation (Data Layer)
@LazySingleton()
class AuthRepositoryImpl implements AuthRepository {
    AuthRepositoryImpl(
        this._remoteDataSource,
        this._localDataSource,
    );

    final AuthRemoteDataSource _remoteDataSource;
    final AuthLocalDataSource _localDataSource;

    @override
    Future<Result<UserData>> login({
        required String userNameOrEmail,
        required String password,
    }) async {
        try {
            final result = await _remoteDataSource.login(
                userNameOrEmail: userNameOrEmail,
                password: password,
            );

            if (result.isSuccess) {
                // Cache user data locally
                await _localDataSource.cacheUserData(result.data);
            }
        }
    }
}

```

```

        return result;
    } catch (e) {
        return Result.failure(e.toString());
    }
}
}

```

Data Sources

```

// Remote Data Source
@LazySingleton()
class AuthRemoteDataSource {
    AuthRemoteDataSource(this._dioClient);
    final DioClient _dioClient;

    Future<Result<UserData>> login({
        required String userNameOrEmail,
        required String password,
    }) async {
        try {
            final response = await _dioClient.post(
                '/Auth/login',
                data: {
                    'userNameOrEmail': userNameOrEmail,
                    'password': password,
                },
            );

            return Result.success(UserData.fromJson(response.data));
        } catch (e) {
            return Result.failure(e.toString());
        }
    }
}

// Local Data Source
@LazySingleton()
class AuthLocalDataSource {
    AuthLocalDataSource(this._sharedPreferences);
    final SharedPreferences _sharedPreferences;

    Future<Result<String?>> getCachedAuthToken() async {

```

```
    final token = _sharedPreferences.getString('auth_token');  
    return Result.success(token);  
  }  
}
```

Kod Yapısı

Klasör Organizasyonu

```
lib/  
├─ config/                # App configuration  
│   ├─ app_routes.dart    # Route definitions  
│   ├─ app_theme.dart     # Theme configuration  
│   └─ environment.dart   # Environment settings  
├─ core/                  # Core infrastructure  
│   ├─ base/              # Base classes and mixins  
│   ├─ cache/             # Caching system  
│   ├─ constants/         # App constants  
│   ├─ di/                # Dependency injection  
│   ├─ error/             # Error handling  
│   ├─ network/           # Network layer  
│   └─ utils/             # Core utilities  
├─ data/                  # Data layer  
│   ├─ datasources/       # API and local data sources  
│   └─ repositories/     # Repository implementations  
├─ domain/                # Domain layer  
│   └─ entities/         # Business entities  
├─ presentation/         # Presentation layer  
│   ├─ providers/         # State management providers  
│   ├─ screens/           # UI screens  
│   └─ widgets/          # Reusable widgets  
├─ models/                # Data models (legacy)  
├─ services/              # Legacy services (deprecated)  
└─ main.dart              # App entry point
```

Naming Conventions

- **Classes:** PascalCase (`AuthProvider` , `EventRepository`)
- **Files:** snake_case (`auth_provider.dart` , `event_repository.dart`)

- **Variables:** camelCase(`userName` , `isLoading`)
 - **Constants:** UPPER_SNAKE_CASE(`API_BASE_URL` , `DEFAULT_TIMEOUT`)
-



Geliştirme Rehberi

Yeni Feature Ekleme

1. Entity Oluştur

```
// lib/domain/entities/my_entity.dart
@freezed
class MyEntity with _$MyEntity {
  const factory MyEntity({
    required String id,
    required String name,
  }) = _MyEntity;

  factory MyEntity.fromJson(Map<String, dynamic> json) =>
    _MyEntityFromJson(json);
}
```

2. DataSource Oluştur

```
// lib/data/datasources/my_entity_remote_data_source.dart
@LazySingleton()
class MyEntityRemoteDataSource {
  MyEntityRemoteDataSource(this._dioClient);
  final DioClient _dioClient;

  // API methods
}
```

3. Repository Oluştur

```
// lib/data/repositories/my_entity_repository.dart
@LazySingleton()
class MyEntityRepository {
  MyEntityRepository(this._remoteDataSource);
  final MyEntityRemoteDataSource _remoteDataSource;
```

```
// Business logic
}
```

4. Provider Oluştur

```
// lib/presentation/providers/my_entity_provider.dart
@injectable
class MyEntityProvider extends BaseProvider {
  MyEntityProvider(this._repository);
  final MyEntityRepository _repository;

  // State management
}
```

5. Screen Oluştur

```
// lib/presentation/screens/my_entity_screen.dart
class MyEntityScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Consumer<MyEntityProvider>(
      builder: (context, provider, child) {
        // UI implementation
      },
    );
  }
}
```

Code Generation

Yeni entity'ler ekledikten sonra:

```
flutter packages pub run build_runner build --delete-conflicting-outputs
```

Best Practices

1. **Error Handling:** Her API çağrısında proper error handling
2. **Loading States:** UI'da loading ve error state'leri göster
3. **Caching:** Uygun yerlerde caching kullan
4. **Logging:** Development'ta logging, production'da kapat

Test Stratejisi

Test Yapısı

```
test/  
├─ unit/                # Unit tests  
│   └─ core/            # Core layer tests  
│       └─ utils/        # Utility tests  
└─ widget/              # Widget tests  
    └─ widgets/          # Widget tests
```

Test Örnekleri

```
// Unit Test  
void main() {  
  group('AuthProvider Tests', () {  
    late MockAuthRepository mockRepository;  
    late AuthProvider provider;  
  
    setUp(() {  
      mockRepository = MockAuthRepository();  
      provider = AuthProvider(mockRepository);  
    });  
  
    test('should login successfully', () async {  
      // Arrange  
      when(() => mockRepository.login(  
        userNameOrEmail: any(named: 'userNameOrEmail'),  
        password: any(named: 'password'),  
      )).thenAnswer((_) async => Result.success(userData));  
  
      // Act  
      final result = await provider.login(  
        userNameOrEmail: 'test@example.com',  
        password: 'password',  
      );  
  
      // Assert  
      expect(result, true);  
    });  
  });  
}
```

```
        expect(provider.isAuthenticated, true);
    });
});
}
```

Test Çalıştırma

```
# All tests
flutter test

# Specific test file
flutter test test/unit/core/auth_provider_test.dart
```

Build ve Deployment

Development Build

```
flutter run --debug
```

Release Build

```
# Android
flutter build apk --release

# iOS (on macOS)
flutter build ios --release
```

Environment-specific Builds

```
# Development
flutter build apk --dart-define=API_URL=http://dev.api.com

# Production
flutter build apk --dart-define=API_URL=https://api.eventup.com
```

Yaygın Problemler

1. Code Generation Hatası

```
flutter clean
flutter pub get
flutter packages pub run build_runner build --delete-conflicting-outputs
```

2. Dependency Injection Hatası

- `@injectable` annotation'ı kontrol et
- Code generation'ı tekrar çalıştır

3. Build Hatası

- Android SDK ve iOS certificates kontrol et
- `flutter doctor` çalıştır

Debug Modu

```
// Debug logging enable
if (Environment.isDevelopment) {
  print('Debug info: $debugInfo');
}
```

Ek Kaynaklar

- [Flutter Documentation](#)
 - [Clean Architecture](#)
 - [Provider Package](#)
 - [GetIt Package](#)
 - [Injectable Package](#)
-

1. Fork yapın
2. Feature branch oluşturun(`git checkout -b feature/amazing-feature`)
3. Commit yapın(`git commit -m 'Add amazing feature'`)
4. Push yapın(`git push origin feature/amazing-feature`)
5. Pull Request oluşturun

Commit Message Format

`type(scope): description`

`feat(auth): add biometric authentication`

`fix(events): resolve event loading issue`

`docs(readme): update installation instructions`

Bu dokümantasyon güncellenmelidir. Yeni feature'lar eklendikçe veya mimari değişiklikler yapıldıkça bu dokümantasyonu güncelleyin.

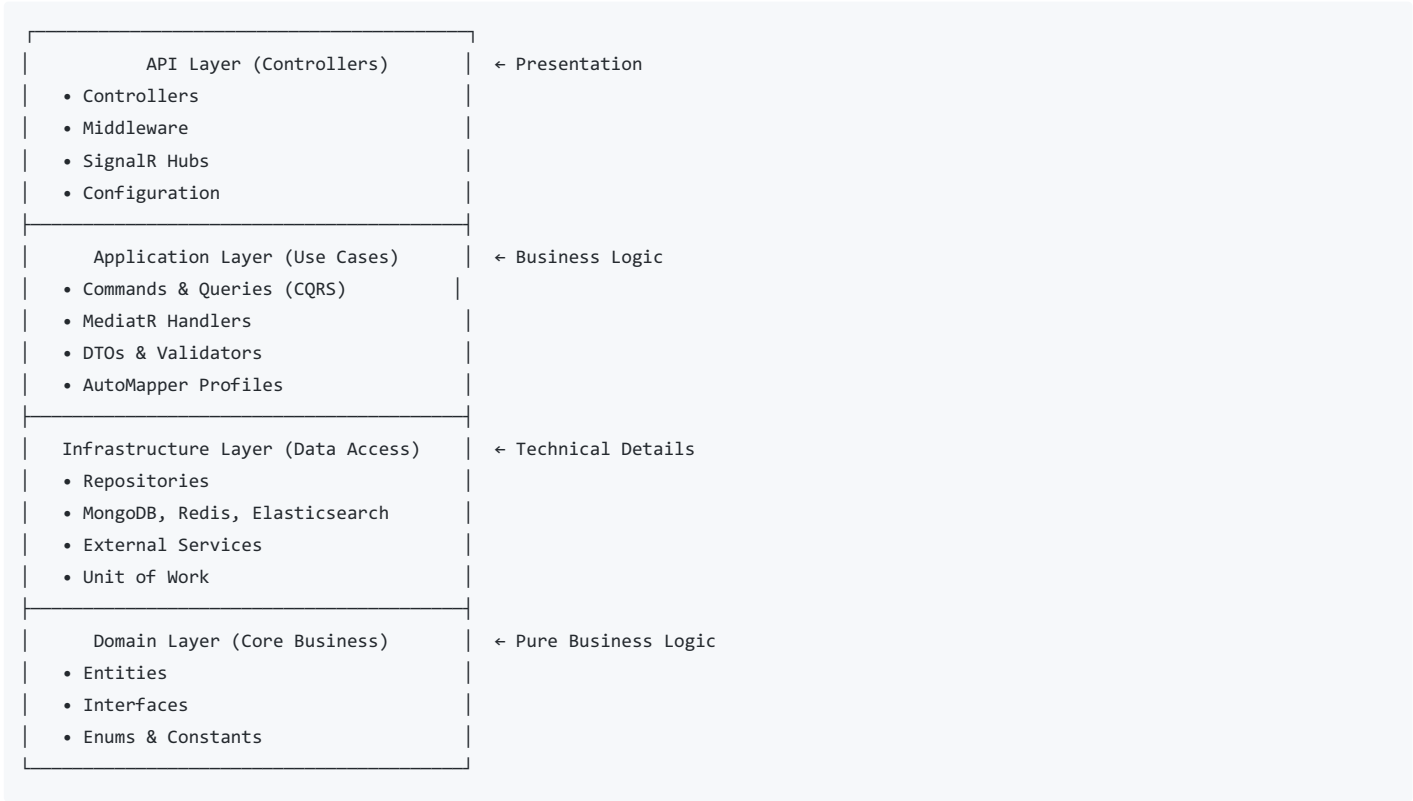
EventUp API Developer Documentation

Bu dokümantasyon EventUp API'sinin UI entegrasyonu için gerekli tüm endpoint'leri, request/response formatlarını ve kullanım örneklerini içerir.

Mimari Yapı

Clean Architecture (Onion Architecture)

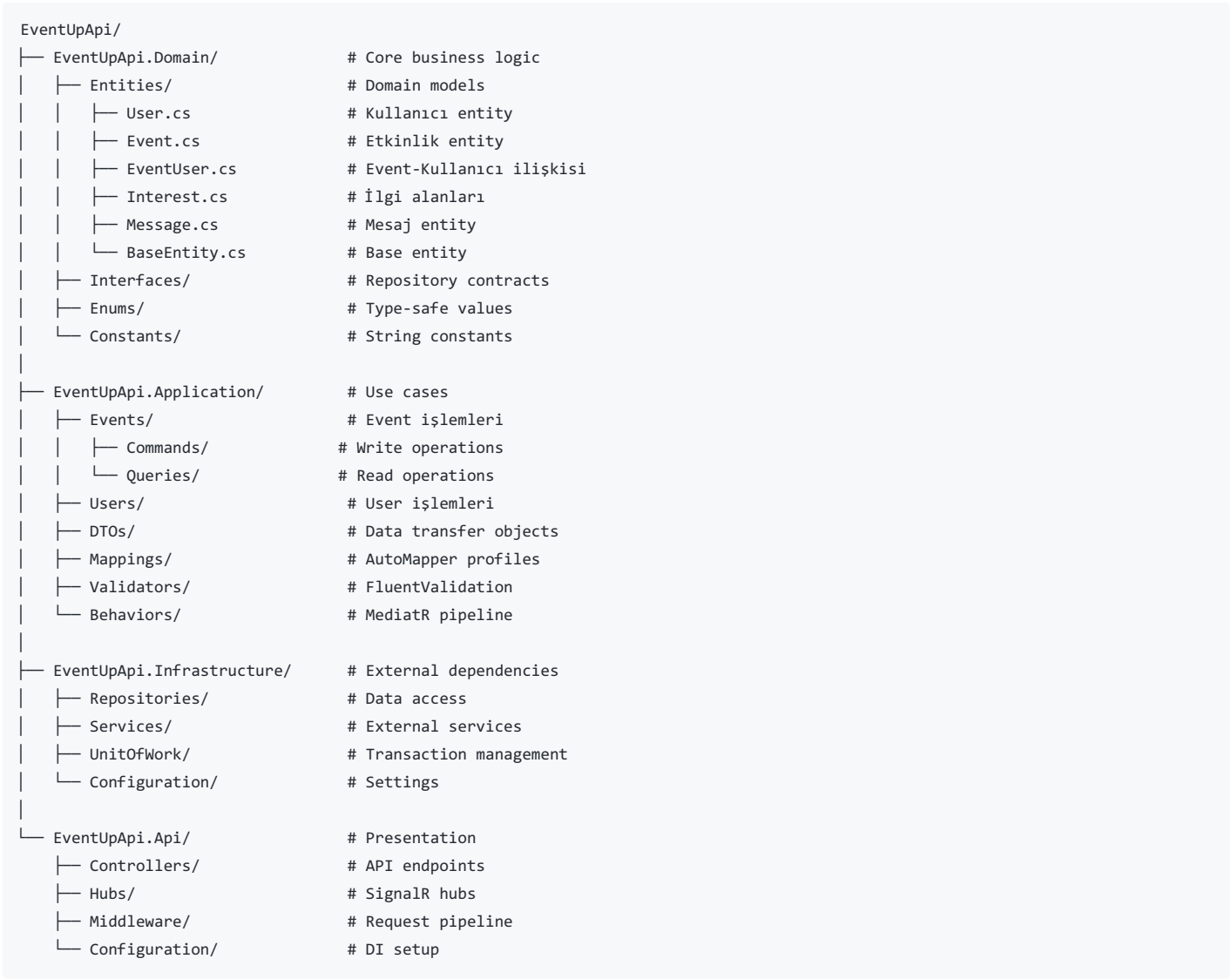
EventUp API, Clean Architecture prensiplerine uygun olarak tasarlanmıştır. Katmanlar arası bağımlılıklar dışı doğru olacak şekilde organize edilmiştir.



Katman Bağımlılıkları

API Layer → Application Layer → Domain Layer
Infrastructure → Application Layer → Domain Layer
Domain Layer → Hiçbir katmana bağımlı değil (Pure)

Proje Yapısı



```
erDiagram
    User {
        string Id
        string UserName
        string Name
        string Surname
        string Email
        string Password
        DateTime LastLoginAt
        DateTime CreatedAt
    }

    Event {
        string Id
        string Title
        string CreatedByUserName
        string Description
        string Location
        DateTime StartDate
        DateTime EndDate
        int Capacity
        string InterestName
        string CoverPhotoUrl
        bool IsLocationConfirmed
        bool IsDateConfirmed
    }

    EventUser {
        string Id
        string EventId
        string UserName
        EventUserStatus Status
        DateTime RequestDate
        DateTime ApprovalDate
        string KickedReason
    }

    Interest {
        string Id
        string Name
    }

    Message {
        string Id
        string MessageGroupId
        string EventId
        string SenderUserName
        string Content
        bool IsRead
    }

    User ||--o{ Event : creates
    Event ||--o{ EventUser : has_participants
    User ||--o{ EventUser : participates
    Interest ||--o{ Event : categorizes
```

Teknoloji Stack

Katman	Teknoloji	Versiyon	Amaç
API	.NET Core	8.0	Web API Framework
Application	MediatR	13.0.0	CQRS Implementation

Application Katman	AutoMapper Teknoloji	12.0.1 Versiyon	Object Mapping Amaç
Application	FluentValidation	11.9.0	Input Validation
Infrastructure	MongoDB	6.0	Primary Database
Infrastructure	Redis	7.0	Distributed Cache
Infrastructure	Elasticsearch	8.11	Log Aggregation
API	SignalR	8.0	Real-time Messaging
API	JWT Bearer	8.0.4	Authentication

Request Pipeline ve Middleware

API request'leri aşağıdaki sırayla işlenir:

- CORS
 - Response Compression & Caching
 - Rate Limiting
 - Serilog Request Logging
 - Authentication & Authorization
 - Global Exception Handling
 - Controllers
 - SignalR Hubs (WebSocket requests)

CQRS Pattern Implementation

Commands (Write Operations):

- Event creation, update, deletion
- User registration, profile updates
- Event participation management

Queries (Read Operations):

- Event filtering and search
- User information retrieval
- Statistics and reporting

Data Flow

- Client Request → API Controller
 - Controller → MediatR → Handler (Command/Query)
 - Handler → Repository (via Unit of Work)
 - Repository → MongoDB/Redis
 - Response ← Repository ← Handler ← Controller ← Client

Caching Strategy

- Redis:** Interest listesi, user sessions
- Response Caching:** Static data, filtered events
- In-Memory:** Frequently accessed data

Authentication Flow

- User Login → JWT Token Generation
 - Token included in Authorization header
 - JWT Middleware validates token
 - User context extracted from token
 - Authorization checks performed

Base URL

https://your-domain.com/api

Authentication

API, JWT (JSON Web Token) tabanlı authentication kullanır. Çoğu endpoint için **Authorization** header'ında Bearer token gereklidir.

Authorization: Bearer <your-jwt-token>

Response Format

Tüm API response'ları aşağıdaki standart formatı kullanır:

```
{
  "success": true,
  "data": { /* response data */ },
  "message": "Operation successful",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

Error response formatı:

```
{
  "success": false,
  "data": null,
  "message": "Error message",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1. Authentication Endpoints

1.1 Register User

Endpoint: POST /api/auth/register **Authentication:** Not required

Request Body:

```
{
  "userName": "johndoe",
  "name": "John",
  "surname": "Doe",
  "email": "john@example.com",
  "password": "SecurePassword123!"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "user123",
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "lastLoginAt": null,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "User registered successfully",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1.2 Login User

Endpoint: POST /api/auth/login **Authentication:** Not required

Request Body:

```
{
  "userNameOrEmail": "johndoe",
  "password": "SecurePassword123!"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "user123",
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Login successful",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1.3 Get Current User

Endpoint: GET /api/auth/me **Authentication:** Required

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Current user information retrieved successfully",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2. Event Endpoints

2.1 Create Event

Endpoint: POST /api/v1.0/eventowner/create **Authentication:** Required

Request Body:

```
{
  "title": "Tech Meetup 2024",
  "description": "Great tech event for developers",
  "location": "Istanbul, Turkey",
  "startDate": "2024-02-15T18:00:00Z",
  "endDate": "2024-02-15T22:00:00Z",
  "capacity": 50,
  "interestName": "Technology",
  "coverPhotoUrl": "https://example.com/photo.jpg",
  "isLocationConfirmed": true,
  "isDateConfirmed": true
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Tech Meetup 2024",
    "createdByUserName": "johndoe",
    "description": "Great tech event for developers",
    "location": "Istanbul, Turkey",
    "startDate": "2024-02-15T18:00:00Z",
    "endDate": "2024-02-15T22:00:00Z",
    "capacity": 50,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Event başarıyla oluşturuldu",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.2 Get Event by ID

Endpoint: POST /api/v1.0/eventcommon/get-by-id **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Tech Meetup 2024",
    "createdByUserName": "johndoe",
    "description": "Great tech event for developers",
    "location": "Istanbul, Turkey",
    "startDate": "2024-02-15T18:00:00Z",
    "endDate": "2024-02-15T22:00:00Z",
    "capacity": 50,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Event başarıyla getirildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.3 Get Filtered Events

Endpoint: POST /api/v1.0/eventcommon/filtered **Authentication:** Required

Request Body:

```
{
  "search": "tech",
  "location": "Istanbul"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Filtrelenmiş etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.4 Update Event

Endpoint: PUT /api/v1.0/eventowner/update **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "title": "Updated Tech Meetup 2024",
  "description": "Updated description",
  "location": "Ankara, Turkey",
  "startDate": "2024-02-20T18:00:00Z",
  "endDate": "2024-02-20T22:00:00Z",
  "capacity": 75,
  "interestName": "Technology",
  "coverPhotoUrl": "https://example.com/new-photo.jpg",
  "isLocationConfirmed": true,
  "isDateConfirmed": true
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Updated Tech Meetup 2024",
    "createdByUserName": "johndoe",
    "description": "Updated description",
    "location": "Ankara, Turkey",
    "startDate": "2024-02-20T18:00:00Z",
    "endDate": "2024-02-20T22:00:00Z",
    "capacity": 75,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/new-photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T11:00:00Z"
  },
  "message": "Event başarıyla güncellendi",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

2.5 Delete Event

Endpoint: DELETE /api/v1.0/eventowner/delete **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "message": "Event başarıyla silindi"
  },
  "message": "Event başarıyla silindi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

3. Event Participation Endpoints

3.1 Join Event

Endpoint: POST /api/v1.0/eventparticipant/join **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Pending",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": null,
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Etkinliğe başarıyla katılım talebi gönderildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

3.2 Approve Participant

Endpoint: PUT /api/v1.0/eventowner/approve-participant **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Approved",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": "2024-01-15T11:00:00Z",
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Katılımcı başarıyla onaylandı",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

3.3 Reject Participant

Endpoint: PUT /api/v1.0/eventowner/reject-participant **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Rejected",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": "2024-01-15T11:00:00Z",
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Katılımcı başarıyla reddedildi",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

3.4 Leave Event

Endpoint: DELETE /api/v1.0/eventparticipant/leave **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "message": "Etkinlikten başarıyla ayrıldınız"
  },
  "message": "Etkinlikten başarıyla ayrıldınız",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

3.5 Get Approved Participants

Endpoint: POST /api/v1.0/eventcommon/approved-participants **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "id": "eventuser123",
      "eventId": "event123",
      "userName": "johndoe",
      "status": "Approved",
      "requestDate": "2024-01-15T10:30:00Z",
      "approvalDate": "2024-01-15T11:00:00Z",
      "kickedDate": null,
      "kickedReason": null
    }
  ],
  "message": "Onaylanmış katılımcılar başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4. User Endpoints

4.1 Get User Basic Info

Endpoint: POST /api/v1.0/user/basic-info **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Kullanıcı temel bilgileri başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.2 Get User Created Events

Endpoint: POST /api/v1.0/eventcommon/created-by-user **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Kullanıcının oluşturduğu etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.3 Get User Participated Events

Endpoint: POST /api/v1.0/eventcommon/participated-events **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event456",
      "title": "Past Event",
      "createdByUserName": "janedoe",
      "description": "Event that already happened",
      "location": "Ankara, Turkey",
      "startDate": "2023-12-15T18:00:00Z",
      "endDate": "2023-12-15T22:00:00Z",
      "capacity": 30,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/past-photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2023-12-10T10:30:00Z",
      "updatedAt": "2023-12-10T10:30:00Z"
    }
  ],
  "message": "Kullanıcının katıldığı etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.4 Get User Upcoming Events

Endpoint: POST /api/v1.0/eventcommon/upcoming-events **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Kullanıcının katılacağı etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

5. Interest Endpoints

5.1 Get All Interests

Endpoint: `GET /api/v1.0/interest` **Authentication:** Required

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "id": "interest1",
      "name": "Technology",
      "description": "Tech related events"
    },
    {
      "id": "interest2",
      "name": "Sports",
      "description": "Sports and fitness events"
    }
  ],
  "message": "Interests retrieved successfully",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

5.2 Create Interest

Endpoint: `POST /api/v1.0/interest` **Authentication:** Required

Request Body:


```
{
  "name": "Art",
  "description": "Art and culture events"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "interest3",
    "name": "Art",
    "description": "Art and culture events"
  },
  "message": "Interest başarıyla oluşturuldu",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

6. Error Handling

Common HTTP Status Codes:

- 200 OK - Success
- 400 Bad Request - Invalid request data
- 401 Unauthorized - Missing or invalid authentication
- 404 Not Found - Resource not found
- 500 Internal Server Error - Server error

Error Response Example:

```
{
  "success": false,
  "data": null,
  "message": "Event not found",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

7. Rate Limiting

API rate limiting kullanır. Request limitleri aşılması durumunda 429 Too Many Requests response döner.

8. SignalR Real-time Communication

Message Hub Connection

WebSocket hub endpoint: /messageHub

Connection URL:

```
wss://your-domain.com/messageHub?access_token=<jwt-token>
```

JavaScript Example:

```
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/messageHub", {
        accessTokenFactory: () => {
            return localStorage.getItem('token');
        }
    })
    .build();

connection.start().then(() => {
    console.log("Connected to message hub");
});
```

9. Testing

Health Check Endpoints:

1. **Simple Health Check:**

```
GET /health/ready
```

2. **Detailed Health Check:**

```
GET /health/live
```

3. **Full Health Check:**

```
GET /health
```

10. UI Integration Notes

Token Management:

1. Login/Register sonrası `token` field'ını `localStorage`'da saklayın
2. Her API request'inde `Authorization: Bearer <token>` header'ını ekleyin
3. Token expire olduğunda kullanıcıyı login sayfasına yönlendirin

Error Handling:

1. `success: false` olan response'ları handle edin
2. Network error'ları için retry mechanism implement edin
3. Loading state'leri gösterin

Real-time Updates:

1. SignalR connection kurarak real-time bildirimler alın
2. Connection lost durumlarını handle edin
3. Automatic reconnection implement edin

Bu dokümantasyon EventUp API'sinin ana endpoint'lerini kapsar. Daha detaylı bilgi için API'nin Swagger dokümantasyonunu kullanabilirsiniz (Development ortamında `/swagger` endpoint'i).